RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

CS23633 – Cloud Computing
Laboratory Record Note Book

Name : _____

Year / Branch / Section : _____

University Register No. : _____

College Roll No. : _____

Semester : _____

Academic Year : _____

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23633 – Cloud Computing

NAME	**Monish D Y**
ROLL NO	**230701195**
YEAR	**III**
DEPT	**CSE**

RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……

Academic Year: …… Semester: …….. Branch: ……...

Register No.

Certified that this is the bonafide record of work done by the above student in the……

Laboratory during the academic year 2026 – 2027.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……..

Internal Examiner External Examiner

INDEX

S. No.	Experiment	Date	GitHub
1	Private Cloud Using OpenStack		
2	Public Cloud: Develop A Simple Application Using PaaS		
3	Hybrid Cloud		
4	Backup Automation Using Azure Blob Storage		
5	VPS / Virtual Machine Setup and Linux Configuration		

EXPT NO. 1 – PRIVATE CLOUD USING OPENSTACK

Aim:

To design, install, and configure a Private Cloud environment using a single-node OpenStack (MicroStack) implementation, and to understand the basic cloud services such as compute, networking, and storage by deploying and accessing virtual machine instances on the private cloud.

OPENSTACK:

Theory:

What is OpenStack?

1. OpenStack is a free, open standard cloud computing platform.
2. It is mostly deployed as Infrastructure-as-a-Service (IaaS).
3. OpenStack allows to deploy virtual machines (VMs) on demand.

It controls large pools of:

- Compute
- Storage
- Networking resources of a datacentre

Managed by admins via dashboard – provides resources to users via web interface.

Private/Hybrid cloud → OpenStack
Public cloud → AWS, Azure...

What is MicroStack?

- MicroStack runs OpenStack locally with minimal setup (single-node).
- Ideal for learning, testing, or small development projects.
- Easy to deploy and test OpenStack on a single machine (laptop or small server).

OpenStack Services:

1. Nova (Compute)
2. Neutron (Networking)
3. Keystone (Identity)

4. Glance (Images)
5. Horizon (Dashboard)

Function	OpenStack	AWS	Description
Compute (VMs)	Nova	EC2	Run virtual machines
Image Management	Glance	AMI	Store and manage VM images
Identity & Access	Keystone	IAM	User authentication, authorization, roles
Networking	Neutron	VPC, SecurityGrp	Manage networks, routers, IPs, firewalls
Block Storage	Cinder	EBS	Persistent volumes for VMs
Object Storage	Swift	S3	Store and retrieve unstructured data
Dashboard (UI)	Horizon	AWS Console	Web interface for managing cloud resources

Procedure:

Installation Steps (Ubuntu 22.04 LTS):

sudo snap install microstack –beta
snap list microstack
sudo apt install python-is-python3
sudo microstack init --auto –control

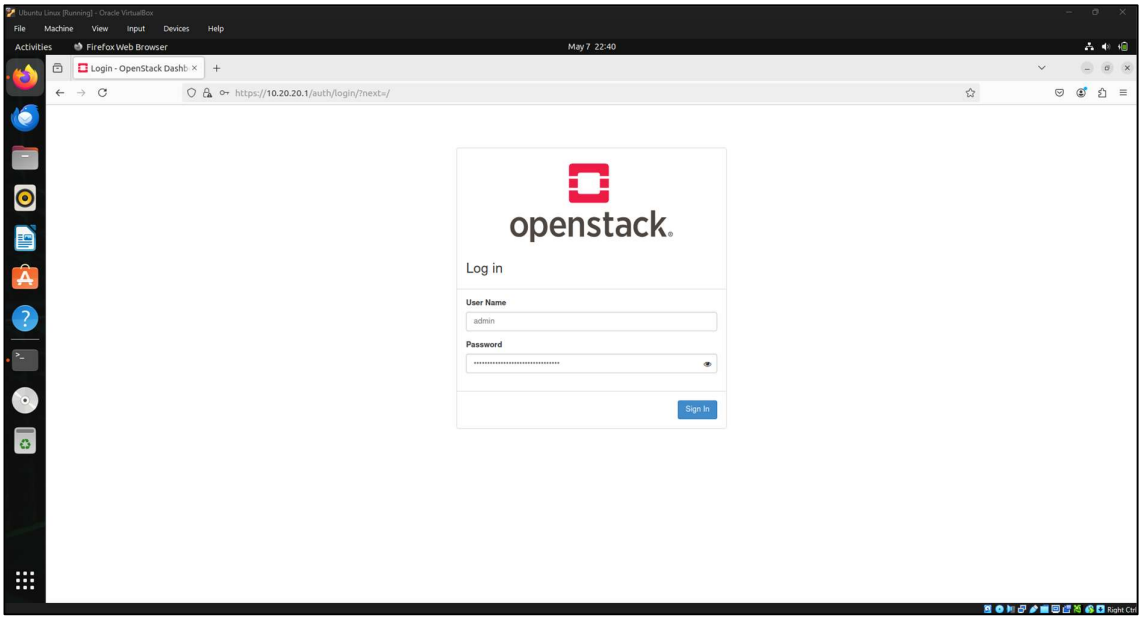
Open browser and navigate to: 10.20.20.1 — the dashboard will appear.

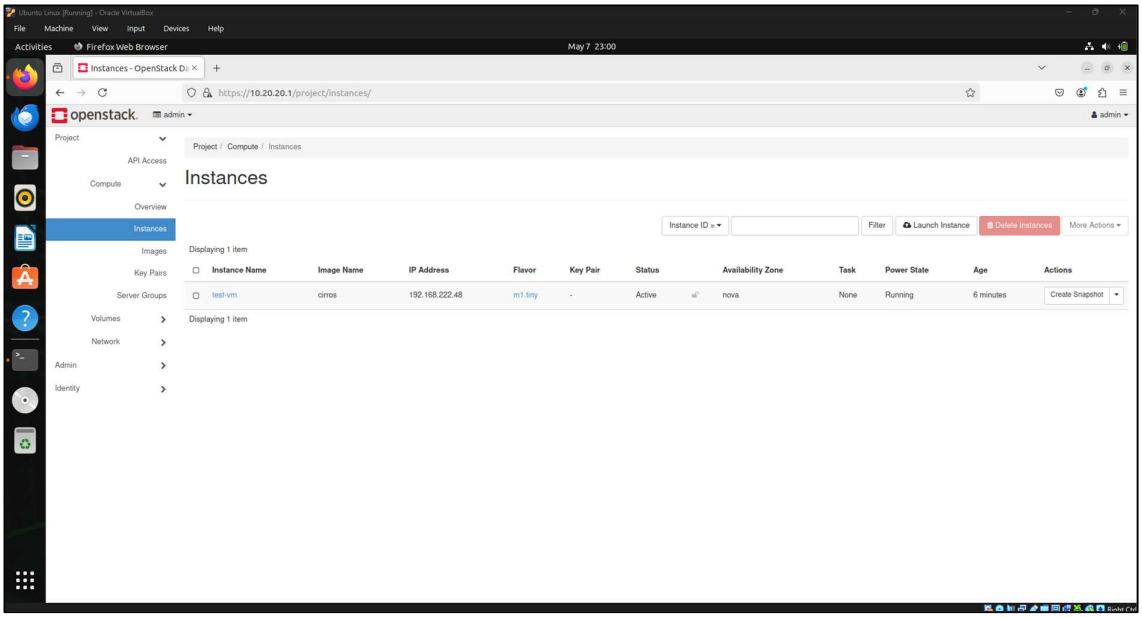
Username: admin

To retrieve the password:

sudo snap get microstack config.credentials.keystone-password

Output:





Result:

A single-node Private Cloud environment was successfully set up using OpenStack (MicroStack). The OpenStack services were properly installed and configured, and the private cloud was able to launch, manage, and access virtual machine instances with functional compute, networking, and storage services, thereby meeting the objectives of the experiment.

EXPT NO. 2 – PUBLIC CLOUD: DEVELOP A SIMPLE APPLICATION USING PAAS

Aim:

To develop and deploy a simple web application on Microsoft Azure App Service to understand Platform as a Service (PaaS).

Procedure:

STEP 1: Create Azure Student Free Account

- Open: https://portal.azure.com
- Login with Microsoft account.
- Activate **Azure for Students**
- After activation → Azure Portal Dashboard opens.

STEP 2: Create a Simple Web Application

- Create a folder on your PC:

  azure-app

- **Create file:** *index.html*

```
<!DOCTYPE html>
<html>
<head>
  <title>My Azure PaaS App</title>
</head>
<body>
  <h1>Hello from Microsoft Azure PaaS!</h1>
  <p>This app is deployed using Azure App Service.</p>
</body>
</html>
```

STEP 3: Create App Service in Azure (PaaS)

- Go to Azure Portal → Click **Create a resource**
- Search → **App Service**
- Click → Create

Fill Details:

Project Details

- Subscription: Azure for Students
- Resource Group: Create new → RG-PaaS-Lab

Instance Details

- App name: my-paas-app-12345 (must be unique)
- Publish: Code
- Runtime stack: Choose .NET / Node.js / Python (For HTML → choose .NET or Node.js)
- Region: Central India (or nearby)

Pricing Plan

- Click Change size → Select **Free (F1)**

Click → Review + Create → Create

Wait 1–2 minutes.

STEP 4: Deploy the Application (Upload Code)

Method 1 (Easy): Deployment Center (ZIP Upload)

- Open your App Service
- Click → Deployment Center
- Select → ZIP Deploy
- Zip your folder azure-app
- Upload ZIP file

Method 2 (Alternative): App Service Editor

- Go to App Service → Advanced Tools (Kudu)
- Click → Go
- Open Debug Console → CMD
- Go to site/wwwroot
- Upload index.html

STEP 5: Run the Application

- Go to App Service → Overview
- Copy the URL:
  Example: *https://my-paas-app-12345.azurewebsites.net*
- Paste in browser

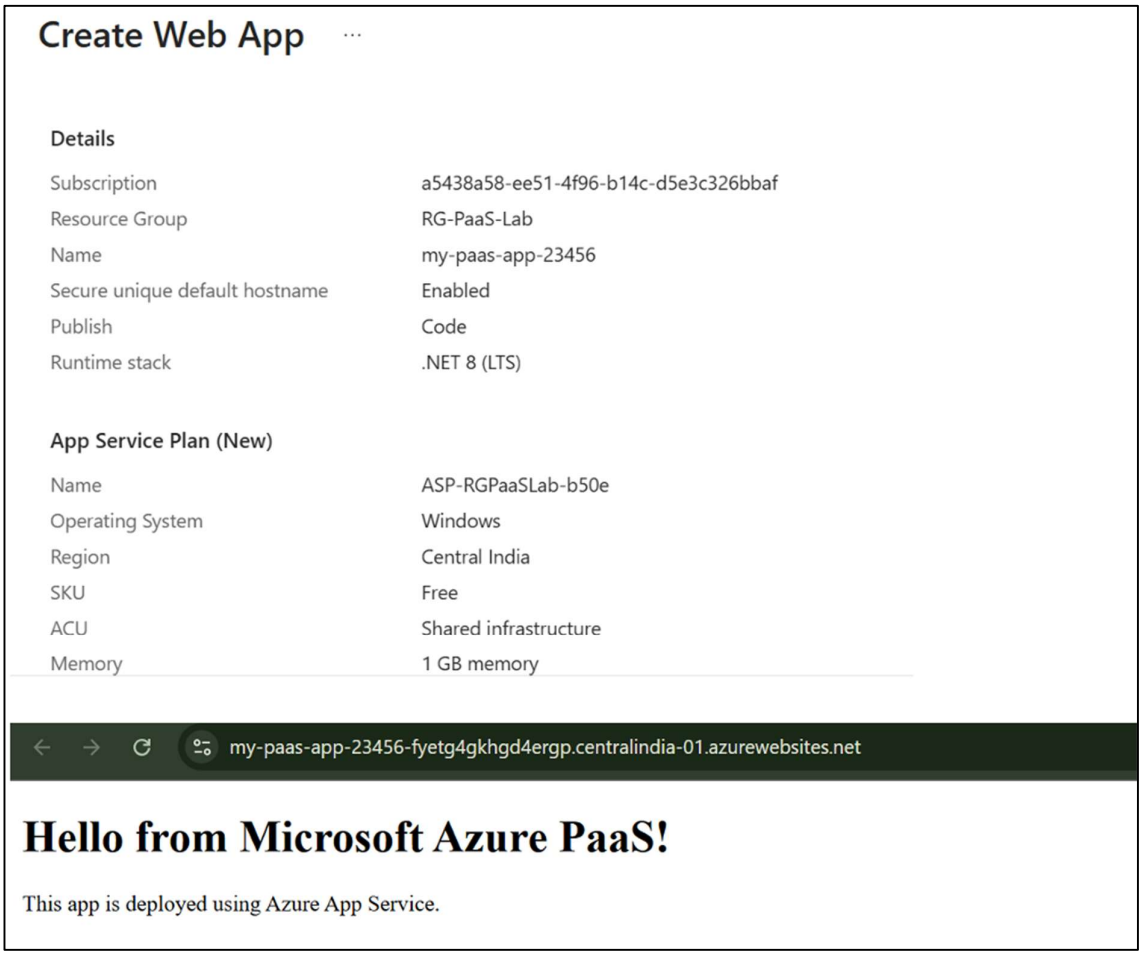
STEP 6: Verify PaaS Concept

Observe:

- No server installation
- No OS configuration
- Azure manages infrastructure
- Only code is deployed

This proves PaaS.

Output:



Result:

The simple web application was successfully deployed using Microsoft Azure App Service, demonstrating the concept of Platform as a Service (PaaS).

EXPT NO. 3 – HYBRID CLOUD

Aim:

To set up a secure VPN connection between an on-premises data center and a public cloud provider such as AWS or Azure, and configure a hybrid network topology using virtual private clouds (VPCs), subnets, and route tables.

Steps Overview:

- Create a Virtual Network (VNet)
- Create Subnets
- Create a VPN Gateway
- Simulate an "On-Prem" network
- Connect them using Site-to-Site VPN

Procedure:

STEP 1: Create a Resource Group

Think of Resource Group as a folder.

1. Login to Azure Portal
2. In search bar type Resource groups
3. Click → Create
4. Fill:
   - Subscription → Your student subscription
   - Resource group name → Hybrid-RG
   - Region → Choose same region everywhere (example: Central India)
5. Click Review + Create
6. Click Create

STEP 2: Create Virtual Network (VNet)

VNet = Your cloud network.

1. Search Virtual Networks
2. Click Create
3. Fill:

Basics Tab

- Resource Group → Hybrid-RG
- Name → Cloud-VNet
- Region → SAME region as resource group

Click Next

IP Addresses Tab

- Address Space: 10.1.0.0/16
- Remove default subnet.

Click + Add Subnet

Create Subnet 1:

- Name: App-Subnet
- Address range: 10.1.1.0/24

Click Add

IMPORTANT: Add Gateway Subnet

Click + Add Subnet again:

- Name: GatewaySubnet (must be EXACT name)
- Address range: 10.1.255.0/27

Click Add → Click Review + Create → Click Create

VNet created.

STEP 3: Create Virtual Network Gateway (VPN Gateway)

This is the Azure side VPN device.

1. Search → Virtual network gateways
2. Click → Create

Fill Details – Basics:

- Resource Group → Hybrid-RG
- Name → Azure-VPN-Gateway
- Region → Same region
- Gateway Type → VPN
- VPN Type → Route-based
- SKU → Select VpnGw1 (basic supported)

- Generation → Generation1

Virtual Network Section
Select: Cloud-VNet

Public IP Address
Click Create new

Name it: VPN-Gateway-IP

Click OK → Click Review + Create → Click Create

IMPORTANT: This takes 30–45 minutes to deploy. Be patient.

STEP 4: Simulate On-Prem Network
Since you don't have a real data center, we simulate. We create a Local Network Gateway (represents on-prem).

1. Search → Local Network Gateways
2. Click → Create

Fill:

- Resource Group → Hybrid-RG
- Name → OnPrem-Gateway
- Region → Same region

IP Address:
Put any dummy public IP (example): 1.1.1.1

Address Space:
Add: 192.168.1.0/24

Click Review + Create → Click Create

Done.

STEP 5: Create VPN Connection
Now connect Azure to On-Prem.

1. Go to Virtual Network Gateways
2. Click your gateway → Azure-VPN-Gateway
3. On left side click Connections
4. Click + Add

Fill:

- Connection Name → Hybrid-Connection
- Connection Type → Site-to-Site (IPsec)
- Virtual Network Gateway → Auto selected
- Local Network Gateway → OnPrem-Gateway
- Shared Key → Type simple password: Azure123

Click OK

VPN connection created.

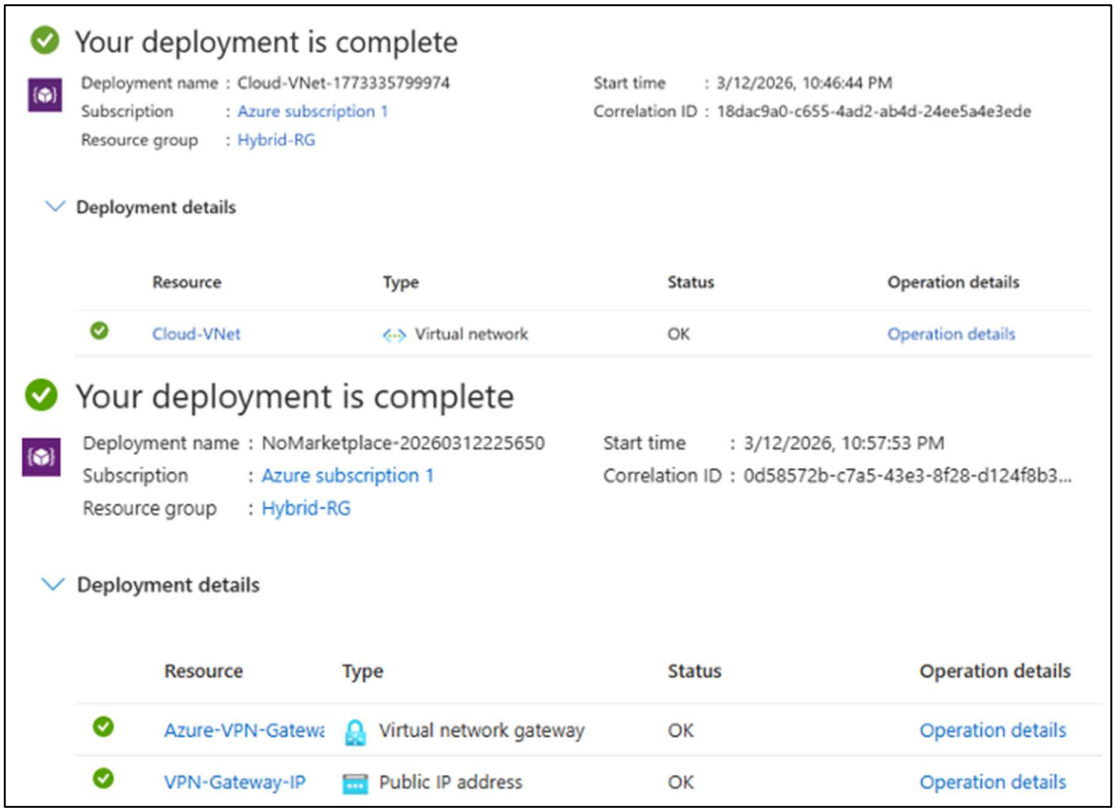
STEP 6: Verify Connection

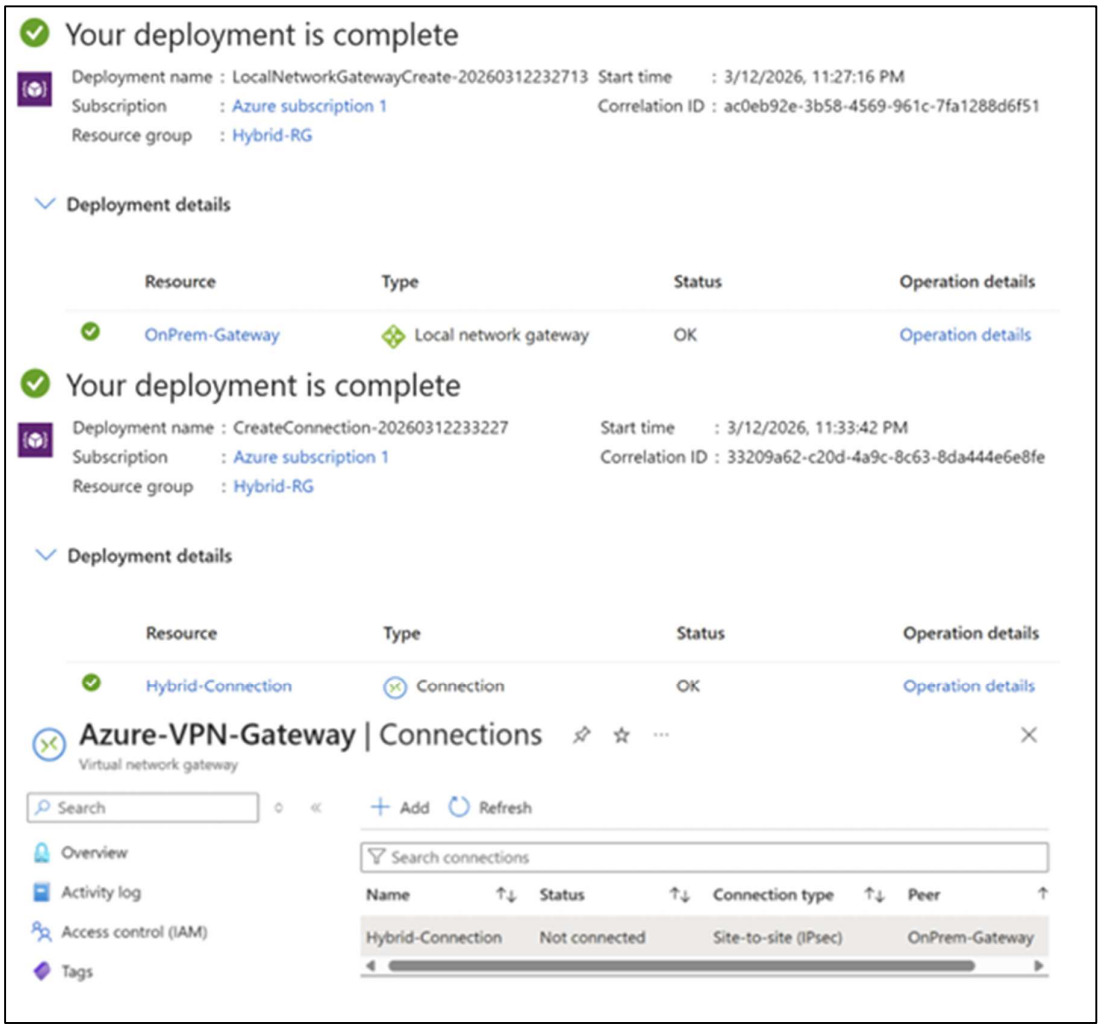
1. Open Azure-VPN-Gateway
2. Click Connections
3. Check Status

If configured with real on-prem router → it will show Connected.

Since we used dummy IP, it will show Not Connected.

<u>Output:</u>





Result:

A hybrid cloud connectivity was successfully set up using Azure Virtual Network Gateway and a simulated on-premises Local Network Gateway connected via a Site-to-Site IPsec VPN tunnel.

EXPT NO. 4 – BACKUP AUTOMATION USING AZURE BLOB STORAGE

Aim:

To design and implement an automated backup solution using Azure Blob Storage and scheduling tools.

Theory:

Azure Blob Storage is a cloud-based object storage service used to store large amounts of unstructured data such as files, backups, logs, and media. It provides high availability, scalability, and secure storage for backup and disaster recovery solutions.

Procedure:

STEP 1: Create Azure Storage Account

1. Login to Azure Portal: https://portal.azure.com
2. Click Create a resource
3. Search Storage Account → Click Create
4. Fill the following details:
   - Subscription: Azure Student Subscription
   - Resource Group: Create new (backup-rg)
   - Storage Account Name: mystoragebackup123
   - Region: South India
   - Performance: Standard
   - Redundancy: Locally Redundant Storage (LRS)
5. Click Review + Create → Create

STEP 2: Create Blob Container

1. Open Storage Account
2. Go to Data Storage → Containers
3. Click + Container

Fill:

- Name: backups, Public Access: Private

4. Click Create

STEP 3: Azure CLI Login

```
az login                # Opens browser for authentication
az account show         # Verify account details
```

STEP 4: Upload Files

```
az storage blob upload-batch \
  --account-name mystoragebackup123 \
  --destination backups \
  --source C:\backup-data

# Uploads all local files to Azure
```

STEP 5: Create Backup Script

```
$source = "C:\backup-data"
$container = "backups"
az storage blob upload-batch `
  --account-name mystoragebackup123 `
  --destination $container `
  --source $source

# Automates backup process
```

STEP 6: Add Timestamp Backup

```
$date = Get-Date -Format "yyyy-MM-dd"
$destination = "backups/$date"
az storage blob upload-batch `
  --account-name mystoragebackup123 `
  --destination $destination `
  --source C:\backup-data

# Stores backup by date
```

STEP 7: Scheduling using Windows Task Scheduler (Detailed)

Steps:

1. Press Windows + R
2. Type: taskschd.msc
3. Press Enter

4. Click Create Basic Task
5. Enter:

- Name: AzureBackupTask
- Description: Daily Azure Backup

6. Click Next

Set Trigger

Choose: Daily

Set time (e.g., 2:00 AM)

Click Next

Set Action

Choose: Start a Program

Configure Program

powershell.exe

Add Arguments:

-ExecutionPolicy Bypass -File "C:\scripts\backup.ps1"

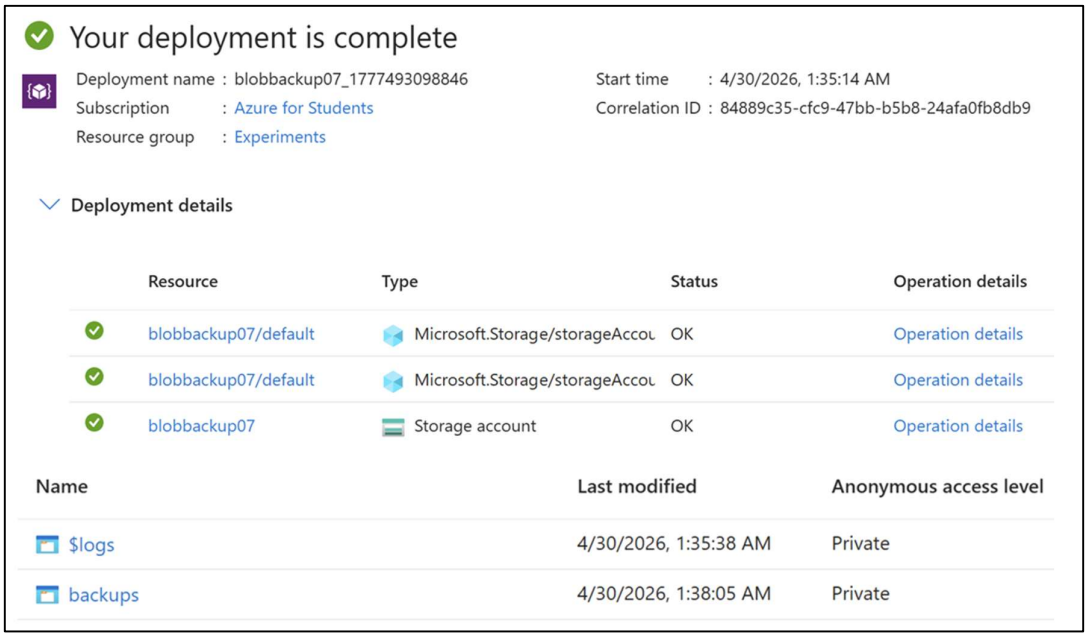
Start In:

C:\scripts

Finish

Click Next → Finish

<u>Output:</u>



```
sanjay [ ~ ]$ az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId":
  "id":
  "isDefault": true,
  "managedByTenants": [],
  "name": "Azure for Students",
  "state": "Enabled",
  "tenantId":
  "user": {
    "cloudShellID": true,
    "name": "live.com#        @rajalakshmi.edu.in",
    "type": "user"
  }
}
```

```
sanjay [ ~ ]$ az storage blob upload-batch \
  --account-name blobbackup07 \
  --destination backups \
  --source ~/backup-data \
  --auth-mode login
Finished[#############################################################]  100.0000%
[
  {
    "Blob": "https://blobbackup07.blob.core.windows.net/backups/file2.txt",
    "Last Modified": "2026-04-29T20:18:54+00:00",
    "Type": "text/plain",
    "eTag": "\"0x8DEA62C88FD4E81\""
  },
  {
    "Blob": "https://blobbackup07.blob.core.windows.net/backups/file1.txt",
    "Last Modified": "2026-04-29T20:18:54+00:00",
    "Type": "text/plain",
    "eTag": "\"0x8DEA62C88FF4823\""
  }
]
```

```
sanjay [ ~ ]$ nano ~/backup.sh
sanjay [ ~ ]$ ./backup.sh
Finished[#############################################################]  100.0000%
[
  {
    "Blob": "https://blobbackup07.blob.core.windows.net/backups/2026-04-29/file2.txt",
    "Last Modified": "2026-04-29T20:21:37+00:00",
    "Type": "text/plain",
    "eTag": "\"0x8DEA62CEA5E94CF\""
  },
  {
    "Blob": "https://blobbackup07.blob.core.windows.net/backups/2026-04-29/file1.txt",
    "Last Modified": "2026-04-29T20:21:37+00:00",
    "Type": "text/plain",
    "eTag": "\"0x8DEA62CEA61C588\""
  }
]
```

2026-04-29						···
file1.txt	4/30/2026, 1:48:54 AM	Hot (Inferred)	Block blob	19 B	Available	···
file2.txt	4/30/2026, 1:48:54 AM	Hot (Inferred)	Block blob	18 B	Available	···

Result:

Successfully implemented automated backup using Azure Blob Storage with scheduling.

EXPT NO. 5 – VPS / VIRTUAL MACHINE SETUP AND LINUX CONFIGURATION

Aim:

To provision a Virtual Private Server (VPS) or create a virtual machine using virtualization software such as VirtualBox or VMware Workstation, install a Linux operating system, and configure network settings including IP address, subnet mask, gateway, and DNS.

Theory:

A Virtual Private Server (VPS) is a virtualized environment that acts like a dedicated server. It can be hosted on cloud platforms or created locally using virtualization tools.

Virtualization software such as:

- VirtualBox
- VMware Workstation

...allows running multiple operating systems on a single physical machine.

Linux distributions like:

- Ubuntu
- CentOS
- Debian

...are commonly used due to stability, security, and open-source nature.

Procedure:

STEP 1: Install Virtualization Software

Option A: Install VirtualBox

1. Download from official website
2. Install with default settings

Option B: Install VMware Workstation

1. Download installer
2. Follow installation wizard

STEP 2: Create Virtual Machine

In VirtualBox:

1. Open VirtualBox → Click New
2. Enter:
   Name: LinuxVM
   Type: Linux
   Version: Ubuntu (64-bit)
3. Allocate:
   RAM: 2 GB (minimum)
   CPU: 1–2 cores
4. Create Virtual Hard Disk:
   Type: VDI
   Size: 20 GB

STEP 3: Attach ISO File

1. Download ISO (e.g., Ubuntu)
2. Go to VM Settings → Storage
3. Attach ISO file

STEP 4: Install Linux OS

1. Start VM
2. Select Install Ubuntu
3. Follow steps:
   Language selection
   Keyboard layout
   Installation type (Normal)
   Create username & password
4. Click Install
5. Restart VM after installation

STEP 5: Configure Network (Automatic - DHCP)

Check IP:

ip a

Check connectivity:

ping google.com

STEP 6: Configure Static IP Address

Edit Netplan config:

sudo nano /etc/netplan/01-netcfg.yaml

Example configuration:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses: [192.168.1.100/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

Apply changes:

sudo netplan apply

STEP 7: Verify Network Configuration

Check IP:

ip a

Check gateway:

ip route

Test DNS:

ping google.com

STEP 8: (Optional) VPS on Cloud

Instead of local VM, you can use:

Microsoft Azure

Amazon EC2

Steps:

1. Create VM instance
2. Select Linux image

3. Configure networking
4. Connect via SSH

Output:

```
pratu@Ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
 qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:cf:28:a5 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86046sec preferred_lft 86046sec
    inet6 fd17:625c:f037:2:1598:1826:d429:8256/64 scope global temporary dynamic
       valid_lft 86291sec preferred_lft 14291sec
    inet6 fd17:625c:f037:2:a00:27ff:fecf:28a5/64 scope global dynamic mngtmpaddr
       valid_lft 86291sec preferred_lft 14291sec
    inet6 fe80::a00:27ff:fecf:28a5/64 scope link
       valid_lft forever preferred_lft forever
```

```
pratu@Ubuntu:~$ ping google.com
PING google.com (172.217.24.14) 56(84) bytes of data.
64 bytes from tsa01s07-in-f14.1e100.net (172.217.24.14): icmp_seq=1 ttl=255 time=
6.36 ms
64 bytes from tsa01s07-in-f14.1e100.net (172.217.24.14): icmp_seq=2 ttl=255 time=
5.32 ms
64 bytes from tsa01s07-in-f14.1e100.net (172.217.24.14): icmp_seq=3 ttl=255 time=
4.94 ms
64 bytes from tsa01s07-in-f14.1e100.net (172.217.24.14): icmp_seq=4 ttl=255 time=
4.82 ms
```

```
pratu@Ubuntu:~$ sudo netplan apply
pratu@Ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
 qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:cf:28:a5 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.50/24 brd 10.0.2.255 scope global noprefixroute enp0s3
       valid_lft forever preferred_lft forever
    inet 10.0.2.15/24 brd 10.0.2.255 scope global secondary dynamic noprefixroute
 enp0s3
```

```
pratu@Ubuntu:~$ ip route
default via 10.0.2.2 dev enp0s3 proto static metric 100
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.50 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.20.20.0/24 dev br-ex proto kernel scope link src 10.20.20.1
```

```
pratu@Ubuntu:~$ ping google.com
PING google.com (142.250.193.174) 56(84) bytes of data.
64 bytes from lcmaaa-ay-in-f14.1e100.net (142.250.193.174): icmp_seq=1 ttl=255 ti
me=4.24 ms
64 bytes from lcmaaa-ay-in-f14.1e100.net (142.250.193.174): icmp_seq=2 ttl=255 ti
me=7.06 ms
64 bytes from lcmaaa-ay-in-f14.1e100.net (142.250.193.174): icmp_seq=3 ttl=255 ti
me=5.56 ms
```

Result:

Successfully created a virtual machine, installed Linux OS, and configured network settings including IP address, gateway, and DNS.